Biblioteca Universitária API

Relatório Consolidado das Features 1, 2, 3 e 4

Aluno: Leonardo da Silva Sousa

Repositório GitHub: https://github.com/LeonardoSSm/bibliotecaapi

Data: 11/03/2026

Objetivo: consolidar as evidências de atendimento dos requisitos dos PDFs das Features 1 a 4.

Sumário

- Feature 1 - TDD: Módulo de Reserva de Livros
- Feature 2 - Integração e Orquestração com APIs externas
- Feature 3 - Modularização e Design de APIs com DTOs
- Feature 4 - Autenticação e Autorização com Spring Security

Feature 1 - TDD

Implementação do módulo de reserva com ciclo Red-Green-Refactor, regras de negócio de disponibilidade, duplicidade e cancelamento, com testes unitários em GREEN.

Feature 2 - Integração Externa

Consumo de Google Books e Open Library via OpenFeign, com orquestração de dados internos e externos e cálculo de score de recomendação.

Feature 3 - Multi-módulos + DTOs

Refatoração para arquitetura multi-módulos (`common-domain`, `external-api-client`, `main-application`) e aplicação de DTOs de entrada e saída.

Feature 4 - Segurança

Autenticação HTTP Basic e autorização por roles (ADMIN/USER), com regras por URL/HttpMethod e validação de cenários 401, 403 e acessos permitidos.

Feature 1

Módulo de Reserva de Livros com TDD

Feature 2

Integração e Orquestração com APIs externas

Feature 3

Modularização e Design de APIs com DTOs

Feature 4

Autenticação e Autorização com Spring Security

Relatorio de Conformidade - Feature 1 (TDD)

Projeto: Biblioteca Universitaria API | **Repo:** git@github.com:LeonardoSSm/bibliotecaapi.git

Commit: c73e8cf | **Data:** 11/03/2026 | **Base:** PDF oficial da Feature 1

1) Resumo da entrega

Funcionalidade: modulo de reserva de livros com TDD, cobrindo criacao/cancelamento de reserva, controle de estoque, duplicidade e erros de negocio.

2) Checklist de requisitos (Feature 1)

Requisito	Status	Evidencia
JUnit 5	ATENDIDO	`ReservaServiceTest.java` com `@Test` e `@DisplayName`.
Estrutura de pacotes de teste correspondente	ATENDIDO	`.../reserva/service/ReservaService.java` e `.../reserva/service/ReservaServiceTest.java`.
Testes da nova funcionalidade em GREEN	ATENDIDO	Surefire: `tests="5" errors="0" failures="0" skipped="0"`.
Pelo menos 1 classe de modelo + 1 classe de servico	ATENDIDO	Modelos: `Reserva`, `ReservaStatus`; Servico: `ReservaService`.
Ciclo Red-Green-Refactor	ATENDIDO	RED (falha inicial), GREEN (passou), REFACTOR (extracao de metodos privados).
README com descricao + 3 a 5 cenarios	ATENDIDO	`README.md` com funcionalidade, integracao, classes e 5 cenarios.
Codigo no GitHub	ATENDIDO	Repositorio publicado em `origin/main`.

3) Evidencia do TDD

RED

```
[ERROR] package br.com...reserva.model does not exist

[ERROR] package br.com...reserva.repository does not exist

[ERROR] cannot find symbol class ReservaService
```

GREEN

```
<testsuite name="br.com...ReservaServiceTest" tests="5" errors="0" failures="0" skipped="0" />
```

REFACTOR

Servico reorganizado com: `buscarLivroPorId`, `buscarLeitorPorId`, `validarReservaDuplicada`, `validarLivroDisponivel`.

4) Trechos de codigo (TDD + regras)

Testes (cenarios principais)

```
@DisplayName("Deve criar reserva ativa com prazo padrão e reduzir quantidade disponível do livro")

@DisplayName("Deve lançar erro ao reservar livro sem quantidade disponível")

@DisplayName("Deve lançar erro ao tentar criar reserva ativa duplicada para o mesmo livro e leitor")

@DisplayName("Deve cancelar reserva ativa e devolver uma unidade para quantidade disponível")

@DisplayName("Deve lançar erro ao cancelar uma reserva inexistente")
```

Servico de negocio

```
public Reserva criarReserva(Long livroId, Long leitorId) {
```

```
    Livro livro = buscarLivroPorId(livroId);
    Leitor leitor = buscarLeitorPorId(leitorId);
    validarReservaDuplicada(livroId, leitorId);
    validarLivroDisponivel(livro, livroId);
    livro.reduzirQuantidadeDisponivel();
    livroRepository.save(livro);
    Reserva reserva = Reserva.criarAtiva(livro, leitor, LocalDate.now(), PRAZO_PADRAO_RESERVA_DIAS);
    return reservaRepository.save(reserva);
}

public Reserva cancelarReserva(Long reservaId) {
    Reserva reserva = reservaRepository.findById(reservaId)
        .orElseThrow(() -> new ReservaNaoEncontradaException(reservaId));
    if (ReservaStatus.ATIVA.equals(reserva.getStatus())) {
        reserva.cancelar();
        Livro livro = reserva.getLivro();
        livro.aumentarQuantidadeDisponivel();
        livroRepository.save(livro);
    }
    return reservaRepository.save(reserva);
}
```

Modelo + repositorio

```
public static Reserva criarAtiva(...) { return new Reserva(..., ReservaStatus.ATIVA, prazoDias); }
public void cancelar() { this.status = ReservaStatus.CANCELADA; }
boolean existsByLivroIdAndLeitorIdAndStatus(Long livroId, Long leitorId, ReservaStatus status);
public void reduzirQuantidadeDisponivel() { ... }
public void aumentarQuantidadeDisponivel() { ... }
```

5) Cenarios (DisplayName + objetivo)

1. Criar reserva ativa com prazo 7 e reduzir estoque.
2. Bloquear reserva quando livro indisponivel.
3. Bloquear reserva ativa duplicada (mesmo livro/leitor).
4. Cancelar reserva ativa e devolver unidade ao estoque.
5. Retornar erro para reserva nao encontrada.

6) Conclusao

Entrega da Feature 1 validada com aderencia ao TDD, cobertura de sucesso/erro, testes verdes e documentacao no README.

Relatorio de Conformidade - Feature 2 (Integracao e Orquestracao)

Projeto: Biblioteca Universitaria API | **Repo:** git@github.com:LeonardoSSm/bibliotecaapi.git

Data da validacao: 11/03/2026 | **Base:** PDF oficial da Feature 2

Escopo: consumo de APIs externas de livros, orquestracao interna e endpoint REST consumivel

1) Resumo da funcionalidade implementada

Foi implementada a funcionalidade de **catalogo enriquecido**, consumindo Google Books e Open Library, cruzando dados com o acervo interno (livro/reserva) e retornando resultado consolidado com `scoreRecomendacao`.

2) Checklist de requisitos da Feature 2

Requisito do PDF	Status	Evidencia objetiva
1. Consumo de pelo menos 1 API externa relevante	**ATENDIDO**	Consumo de `Google Books API` + complemento via `Open Library API`.
2. Criacao de clients de API (OpenFeign recomendado)	**ATENDIDO**	`GoogleBooksClient` e `OpenLibraryClient` com `@FeignClient`.
3. Organizacao em camadas (Service e Controller)	**ATENDIDO**	`LivrosIntegracaoService` (orquestracao/algoritmo) e `LivrosIntegracaoController` (endpoint REST).
4. Algoritmo com transformacao dos dados consumidos	**ATENDIDO**	Calculo de `scoreRecomendacao` com disponibilidade interna, nota externa, volume de avaliacoes, recencia e completude de metadados.
5. Classes para dados brutos e resposta consolidada	**ATENDIDO**	DTOs em `dto.external.google`, `dto.external.openlibrary` e `dto.response`.
6. Criacao de endpoint consumivel	**ATENDIDO**	Endpoint: `GET /api/v1/livros/enriquecidos?query={termo}&max={1..20}`.
7. Consumo pelo projeto da 1a disciplina	**ATENDIDO**	Contrato pronto e documentado no `README.md` com instrucoes de consumo pelo projeto da 1a disciplina.

3) Confirmacao de testes

Suite de testes da Feature 2 validada via Surefire: `LivrosIntegracaoServiceTest - tests="4" errors="0" failures="0" skipped="0"`.

Cenarios cobertos

1. Consolidacao de dados externos com disponibilidade interna e score.
2. Fallback quando Open Library falha.
3. Ordenacao por score e limite de retorno.
4. Validacao de query invalida.

4) Trechos de codigo (evidencias)

4.1 Clients Feign (consumo externo)

```
@FeignClient(name = "googleBooksClient", url = "${app.integracoes.google-books.base-url}")

public interface GoogleBooksClient {

    @GetMapping("/volumes")

    GoogleBooksVolumesResponse buscarVolumes(...);

}


@FeignClient(name = "openLibraryClient", url = "${app.integracoes.open-library.base-url}")

public interface OpenLibraryClient {

    @GetMapping("/search.json")

    OpenLibrarySearchResponse buscarLivros(...);
```

```
}
```

4.2 Service com orquestracao e algoritmo

```
public LivrosEnriquecidosResponse buscarLivrosEnriquecidos(String query, Integer max) {
    GoogleBooksVolumesResponse volumesResponse = googleBooksClient.buscarVolumes(...);
    Map<String, OpenLibraryDoc> openLibraryPorIsbn = buscarOpenLibraryPorIsbn(...);

    List<LivroEnriquecidoResponse> resultados = volumes.stream()
        .map(volume -> montarRespostaEnriquecida(volume, openLibraryPorIsbn))
        .sorted(Comparator.comparing(LivroEnriquecidoResponse::scoreRecomendacao).reversed())
        .limit(limite)
        .toList();

    return new LivrosEnriquecidosResponse(termo, resultados.size(), resultados);
}

private int calcularScore(...) {
    // disponibilidade interna, nota, avaliacoes, recencia e metadados
}
```

4.3 Controller com endpoint consumivel

```
@RestController
@RequestMapping("/api/v1/livros")
public class LivrosIntegracaoController {

    @GetMapping("/enriquecidos")
    public LivrosEnriquecidosResponse buscarLivrosEnriquecidos(
            @RequestParam("query") @NotBlank String query,
            @RequestParam(value = "max", defaultValue = "10") @Min(1) @Max(20) Integer max
    ) { ... }
}
```

4.4 Testes unitarios da Feature 2

```
@DisplayName("Deve consolidar dados externos com disponibilidade interna e calcular score")
@DisplayName("Deve continuar retornando resultados quando Open Library estiver indisponível")
@DisplayName("Deve limitar quantidade de retorno e ordenar por score de recomendação")
@DisplayName("Deve lançar erro quando query estiver em branco")
```

5) Evidencias de configuracao

- `@EnableFeignClients` na classe principal da aplicacao.
- Configuracoes em `application.yml` para base URL do Google Books e Open Library.
- `GOOGLE_BOOKS_API_KEY` opcional por variavel de ambiente.

6) Como demonstrar na apresentacao

1. Executar API: `./mvnw spring-boot:run`.
2. Chamar endpoint: `GET /api/v1/livros/enriquecidos?query=clean%20architecture&max=5`.
3. Mostrar retorno com campos consolidados e `scoreRecomendacao`.
4. Mostrar no README o contrato para consumo pelo projeto da 1a disciplina.

7) Conclusao

A Feature 2 foi implementada em conformidade com os requisitos tecnicos do PDF, com integracao externa real, orquestracao em servico, endpoint REST consumivel, algoritmo de valor agregado e testes unitarios verdes.

Relatorio de Conformidade - Feature 3 (Modularizacao e DTOs)

Projeto: Biblioteca Universitaria API | **Repo:** git@github.com:LeonardoSSm/bibliotecaapi.git

Commit: eabe069 | **Data:** 11/03/2026 | **Base:** PDF oficial da Feature 3

Opcao escolhida: Opcao 1 (Refatoracao do projeto existente para multi-modulos)

1) Resumo da entrega

O projeto foi refatorado para arquitetura multi-modulos com separacao clara de responsabilidades, aplicacao de Request/Response DTOs na API principal e orquestracao de dados internos + externos para respostas orientadas ao cliente.

2) Checklist dos requisitos da Feature 3

Requisito do PDF	Status	Evidencia objetiva
1. Estrutura multi-modulos (common-domain, external-api-client, main-application)	**ATENDIDO**	Modulos Maven criados no projeto raiz com responsabilidades separadas e dependencias entre modulos.
2. Aplicacao de DTOs (Request em POST/PUT e Response em GET/POST/PUT)	**ATENDIDO**	`LivroRequestDTO` usado em `POST/PUT /api/v1/livros` e `LivroResponseDTO` retornado em `GET/POST/PUT`.
3. Encapsulamento do dominio (sem validacoes HTTP em entidades)	**ATENDIDO**	Entidades do modulo `common-domain` usam anotacoes JPA, sem `@NotBlank`, `@Email` etc.
4. Orquestracao de dados internos e externos para Response DTO	**ATENDIDO**	`LivrosIntegracaoService` combina Google Books + Open Library + dados internos para gerar `LivrosEnriquecidosResponse`.
5. README com explicacao da arquitetura, DTOs e mapeamentos	**ATENDIDO**	`README.md` atualizado com responsabilidades dos modulos, uso de DTOs e fluxo de mapeamento/orquestracao.

3) Evidencia da estrutura multi-modulos

```
biblioteca-api/
  pom.xml (packaging pom)
  common-domain/
    src/main/java/.../livro/model/Livro.java
    src/main/java/.../leitor/model/Leitor.java
    src/main/java/.../reserva/model/Reserva.java
    src/main/java/.../reserva/model/ReservaStatus.java
  external-api-client/
    src/main/java/.../integracao/livros/client/GoogleBooksClient.java
    src/main/java/.../integracao/livros/client/OpenLibraryClient.java
    src/main/java/.../integracao/livros/dto/external/...
  main-application/
    src/main/java/.../controller
    src/main/java/.../service
    src/main/java/.../repository
    src/main/java/.../dto/request
    src/main/java/.../dto/response
```

4) Trechos de codigo (DTOs e mapeamento)

4.1 Request DTO com validacao

```
public record LivroRequestDTO(
    @NotBlank @Size(max = 255) String titulo,
```

```
    @NotBlank @Size(max = 255) String autor,
    @NotBlank @Size(min = 10, max = 17) String isbn,
    @NotBlank @Size(max = 255) String categoria,
    @Min(1) Integer quantidadeTotal,
    @Min(0) Integer quantidadeDisponivel
) { }
```

4.2 Controller usando Request/Response DTOs

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public LivroResponseDTO criarLivro(@Valid @RequestBody LivroRequestDTO requestDTO) {
    return livroApplicationService.criarLivro(requestDTO);
}

@PutMapping("/{id}")
public LivroResponseDTO atualizarLivro(@PathVariable Long id,
                                       @Valid @RequestBody LivroRequestDTO requestDTO) {
    return livroApplicationService.atualizarLivro(id, requestDTO);
}
```

4.3 Service mapeando DTO -> dominio -> DTO

```
public LivroResponseDTO criarLivro(LivroRequestDTO requestDTO) {
    validarQuantidades(requestDTO.quantidadeTotal(), requestDTO.quantidadeDisponivel());

    Livro livro = new Livro(
        null,
        requestDTO.titulo(),
        requestDTO.autor(),
        requestDTO.isbn(),
        requestDTO.categoria(),
        requestDTO.quantidadeTotal(),
        requestDTO.quantidadeDisponivel()
    );

    return mapearParaResponse(livroRepository.save(livro));
}
```

5) Encapsulamento do dominio (common-domain)

As entidades de dominio foram movidas para `common-domain` e nao carregam validacoes HTTP.

```
@Entity
@Table(name = "livros")
public class Livro {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false)
    private String titulo;

    @Column(nullable = false)
    private String autor;

    // ...demais campos de dominio e regras de negocio internas
}
```

6) Orquestracao de dados para Response DTO orientado ao cliente

```
public LivrosEnriquecidosResponse buscarLivrosEnriquecidos(String query, Integer max) {
    GoogleBooksVolumesResponse volumesResponse = googleBooksClient.buscarVolumes(...);
    Map<String, OpenLibraryDoc> openLibraryPorIsbn = buscarOpenLibraryPorIsbn(...);

    List<LivroEnriquecidoResponse> resultados = volumes.stream()
        .map(volume -> montarRespostaEnriquecida(volume, openLibraryPorIsbn))
        .sorted(Comparator.comparing(LivroEnriquecidoResponse::scoreRecomendacao).reversed())
        .limit(limite)
        .toList();

    return new LivrosEnriquecidosResponse(termo, resultados.size(), resultados);
}
```

7) Confirmacao de testes em GREEN

Resultado da suite no modulo principal: `tests="14" errors="0" failures="0" skipped="0"` (BibliotecaApiApplicationTests, ReservaServiceTest, LivrosIntegracaoServiceTest, LivroApplicationServiceTest).

```
TEST-...ReservaServiceTest.xml: tests="5" errors="0" failures="0"
TEST-...LivrosIntegracaoServiceTest.xml: tests="4" errors="0" failures="0"
TEST-...LivroApplicationServiceTest.xml: tests="4" errors="0" failures="0"
TEST-...BibliotecaApiApplicationTests.xml: tests="1" errors="0" failures="0"
```

8) Conclusao

A Feature 3 foi concluida com aderencia aos requisitos do PDF: arquitetura multi-modulos, uso correto de DTOs para entrada/saida, dominio encapsulado e orquestracao de dados para respostas mais uteis ao cliente da API.

Relatorio de Conformidade - Feature 4 (Autenticacao e Autorizacao)

Projeto: Biblioteca Universitaria API | **Repo:** git@github.com:LeonardoSSm/bibliotecaapi.git

Commit base: e7c7ab4 | **Data:** 11/03/2026 | **Base:** PDF oficial da Feature 4

Escopo: HTTP Basic + autorizacao por URL/HttpMethod + seguranca por padrao

1) Resumo da entrega

Foi implementada a camada de seguranca com Spring Security para autenticar usuarios via HTTP Basic e autorizar acesso por roles (`ADMIN` e `USER`) em dois contextos distintos de endpoints, seguindo o principio do menor privilegio e o comportamento de falha segura por padrao.

2) Checklist dos requisitos da Feature 4

Requisito do PDF	Status	Evidencia objetiva
Configurar Spring Security com autenticacao HTTP Basic	**ATENDIDO**	`SecurityConfig.securityFilterChain()` com `httpBasic()`.
Todas as requisicoes exigem autenticacao	**ATENDIDO**	`anyRequest().authenticated()` configurado.
Usuarios em memoria (ADMIN e USER) com senha codificada	**ATENDIDO**	`InMemoryUserDetailsManager` com usuarios `admin` e `user`, senha codificada por `BCryptPasswordEncoder`.
Autorizacao por URL/Request Matching em pelo menos 2 contextos	**ATENDIDO**	Contexto 1: `/api/v1/livros/**` (GET para USER/ADMIN; escrita somente ADMIN). Contexto 2: `/api/v1/livros/enriquecidos` (GET somente ADMIN).
Uso de HttpMethod nas regras	**ATENDIDO**	Regras com `HttpMethod.GET`, `POST`, `PUT`, `PATCH`, `DELETE`.
Seguranca por padrao (rotas nao mapeadas exigem autenticacao)	**ATENDIDO**	Regra global `anyRequest().authenticated()` + teste dedicado de 401.
Documentacao no README e evidencias de testes	**ATENDIDO**	`README.md` atualizado com usuarios, regras por contexto e cenarios de validacao.

3) Trechos de codigo (configuracao de seguranca)

3.1 SecurityFilterChain com autenticacao e autorizacao

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/v1/livros/enriquecidos/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.POST, "/api/v1/livros/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.PUT, "/api/v1/livros/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.PATCH, "/api/v1/livros/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE, "/api/v1/livros/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.GET, "/api/v1/livros/**").hasAnyRole("ADMIN", "USER")
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

3.2 Usuarios em memoria com senha codificada

```
@Bean
UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
    UserDetails admin = User.withUsername("admin")
        .password(passwordEncoder.encode("admin123"))
        .roles("ADMIN")
        .build();

    UserDetails user = User.withUsername("user")
        .password(passwordEncoder.encode("user123"))
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(admin, user);
}

@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

4) Evidencias de testes de seguranca

Classe de teste: `SecurityConfigWebTest` com validacao explicita de cenarios de sucesso e falha (401 sem autenticacao, 403 por role inadequada e acessos permitidos por role correta).

4.1 Cenarios cobertos

1. Sem autenticacao em `GET /api/v1/livros` retorna `401`.
2. `USER` em `GET /api/v1/livros` retorna `200`.
3. `USER` em `POST /api/v1/livros` retorna `403`.
4. `ADMIN` em `POST /api/v1/livros` retorna `201`.
5. `USER` em `GET /api/v1/livros/enriquecidos` retorna `403`.
6. `ADMIN` em `GET /api/v1/livros/enriquecidos` retorna `200`.
7. Sem autenticacao em endpoint nao mapeado retorna `401` (seguranca por padrao).

4.2 Resultado da suite

```
SecurityConfigWebTest: tests="7" errors="0" failures="0" skipped="0"
ReservaServiceTest: tests="5" errors="0" failures="0"
LivrosIntegracaoServiceTest: tests="4" errors="0" failures="0"
LivroApplicationServiceTest: tests="4" errors="0" failures="0"
BibliotecaApiApplicationTests: tests="1" errors="0" failures="0"
TOTAL main-application: tests="21" errors="0" failures="0"
```

5) Principios de seguranca aplicados

- **Menor privilegio:** `USER` so possui leitura de catalogo; escrita e contexto enriquecido ficam com `ADMIN`.
- **Seguranca por padrao:** rotas nao autorizadas explicitamente continuam protegidas por autenticacao.
- **Falha segura:** ausencia de credencial gera `401`; credencial sem permissao gera `403`.

6) Conclusao

A Feature 4 foi implementada com aderencia aos requisitos do PDF, com autenticacao HTTP Basic, autorizacao por roles em dois contextos, regras por HttpMethod, seguranca por padrao e evidencias de testes automatizados em GREEN.